Accessibility Benchmark
Aquavena for the Visually Impaired

Comparing `adriens.github.io/aquavena` vs `aquavena.nc`
Web Accessibility · Low Vision · WCAG 2.1 AA · New Caledonia

Adriens

x.com/rastadidi

May 30, 2026

Abstract

*This report automatically compares the web accessibility of **adriens.github.io/aquavena** — designed for visually impaired users — against the source website **aquavena.nc**, a dietetic meal delivery service in New Caledonia. Standard CLI tools (pa11y-ci, Lighthouse, axe-core, webhint) are used as a common reference. Results show a **100/100 Lighthouse score and 0 WCAG 2.1 AA errors** for our site, versus 91/100 and 67 errors for the official website. All audit data is persisted in a DuckDB database for longitudinal tracking.*

Table of contents

1 Inception

This project was not born in a research lab or a corporate accessibility audit. It started at a kitchen table, from a very simple and concrete situation.

My mother has been a loyal Aquavena customer for years. Every week she orders her meals from their service, and she genuinely enjoys the food. But she is visually impaired — and as her condition evolved, consulting the weekly menus on the official website became increasingly difficult, then impossible to do on her own. The layout, the font sizes, the colour contrasts, the absence of any audio alternative: none of it was designed with her in mind.

The consequence was small but recurring, and that is precisely what made it significant. Every week, she had to wait for someone to be available — a family member, a neighbour — to read the menus out loud to her. She lost the freedom to plan her own meals at her own pace. A minor dependency, perhaps, but one that repeated itself week after week, quietly eroding her autonomy.

I am a software engineer. I realised that a few hours of work could give that autonomy back to her: a dedicated page she could open on her tablet, with large text, high contrast, a dark mode for tired eyes, and a button to have the menus read aloud in her own language. Something she could use alone, at any hour, without asking anyone for help.

That is why this site exists. It is not a product, not a startup, not a statement. It is a small act of care, built with the tools I know, for the person who matters most. The accessibility work documented in this report is a direct consequence of that intent: if the site is meant to help someone with low vision, it had better actually be accessible.

A note to the Aquavena team, should they ever read this: there is no competition here, no monetisation, no hidden agenda. Every menu on this site links back to aquavena.nc to place orders. This is simply a more accessible front door to a service my mother loves.

2 Goals and Broader Vision

2.1 Immediate goal

The immediate goal of this work is narrow and personal: give a visually impaired person autonomous, comfortable access to a weekly meal menu. That problem is solved. But the process of solving it surfaced something more general.

2.2 A replicable methodology

Building this site required answering a question that turns out to be surprisingly hard in practice: *how do you know whether a website is actually accessible?* Subjective assessment does not scale. Manual inspection is inconsistent. What this project demonstrated is that a fully automated, score-based pipeline can answer that question reproducibly and cheaply.

The methodology is as follows. A set of open-source CLI tools — pa11y-ci, Lighthouse, axe-core — each consume a URL and produce a structured output: a JSON file containing a score, a list of violations, and enough metadata to track change over time. Those outputs are loaded into an analytical database (DuckDB), which makes them queryable, comparable, and archivable. A report is then generated automatically from that database.

The key insight is that **accessibility becomes measurable**. Once it is measurable, it can be benchmarked, monitored, and improved in the same way that software performance or test coverage is managed: with targets, regressions, and trends.

2.3 Transferability to other accessibility domains

Low vision is one accessibility challenge among many. The same pipeline — automated audit → structured score → database → report — applies without modification to other user profiles:

- **Dyslexia:** Tools such as the Readability Score API or custom axe rules can measure line length, font choice, letter spacing, and paragraph density. A dyslexic-friendly site has computable characteristics, and a score can be assigned to them just as Lighthouse assigns a score to colour contrast.
- **Motor impairment:** Keyboard navigability, focus order, and touch target size are already audited by Lighthouse and axe-core. A pipeline targeting these specific rules would produce a "motor accessibility score" for any site.
- **Cognitive load:** Metrics such as reading level (Flesch-Kincaid), page complexity, and number of interactive elements per viewport can be computed automatically and tracked over time.

The template is always the same: identify the user profile, map it to computable signals, plug those signals into the pipeline, and let the data speak. The Aquavena project is one instantiation of that template. Others could follow with minimal adaptation — including, potentially, the official aquavena.nc site itself.

3 Site Architecture

3.1 Design principles

The companion site was built around a single constraint: every design decision had to serve a visually impaired user first. Three principles guided the work throughout.

Readability before aesthetics. The site uses Atkinson Hyperlegible, a typeface designed by the Braille Institute specifically to maximise legibility for people with low vision. Each letterform is engineered to be unambiguous: no `l` that looks like `1`, no `O` that looks like `0`. The base font size is 18px, adjustable at runtime via A+ / A− buttons that persist across sessions in `localStorage`.

Multiple access modalities. Users should never be forced to read. Every menu can be listened to via the Web Speech API (text-to-speech), one day at a time or the entire week in sequence. Menus are also exposed as RSS feeds and iCal calendars so that a screen reader, a podcast app, or a calendar assistant can consume them without opening a browser at all.

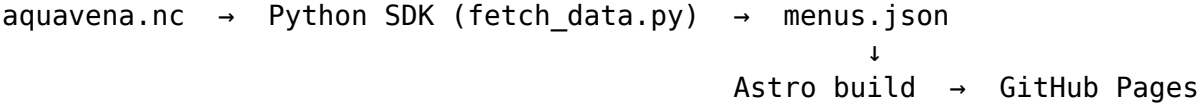
Progressive Web App. The site is installable on any device (iOS, Android, desktop) and works fully offline once cached, so connectivity issues — relevant in parts of New Caledonia — do not block access to the weekly menus.

3.2 Technology stack

Layer	Technology	Role
Framework	Astro 4	Static site generator — zero JS by default
Styling	Tailwind CSS 3	Utility-first CSS, WCAG-compliant colour palette
Typography	Atkinson Hyperlegible	Typeface designed for low vision
Icons	Font Awesome 6	Accessible SVG icons with aria-labels
TTS	Web Speech API	Browser-native text-to-speech, no dependency
PWA	Vite PWA / Workbox	Service worker, offline cache, installability
Data	Python + SDK	Custom scraper — fetches menus from aquavena.nc
Feeds	Astro RSS	RSS 2.0 feeds per diet regime
Calendars	ical.js	iCal (.ics) exports per diet regime
CI/CD	GitHub Actions	Nightly data fetch + build + deploy to GitHub Pages
AI interface	Claude MCP skill	Natural-language menu queries via Model Context Protocol

3.3 Data pipeline

Menus are not hardcoded. A Python SDK scrapes the weekly menus from aquavena.nc each night via a GitHub Actions workflow. The scraped data is written to `src/data/menus.json`, which Astro consumes at build time to generate the static HTML pages. The resulting site is deployed to GitHub Pages with no server-side component.



The site is therefore always up to date by the following morning, with no manual intervention required.

3.4 Accessibility engineering

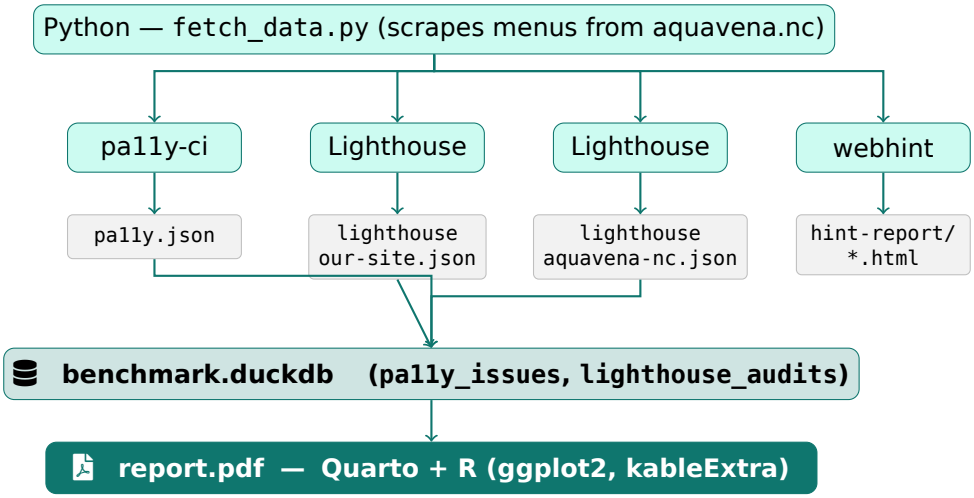
WCAG 2.1 AA conformance was achieved iteratively, guided by three complementary tools run after every change: **pa11y-ci** (WCAG rule violations), **Lighthouse** (scored audit), and **axe-cli** (axe-core engine). The main categories of issues identified and resolved were:

- **Colour contrast (1.4.3):** All interactive elements — speak buttons, tab labels, action links — were systematically audited. Colours such as `bg-orange-500` and `bg-sky-500` were replaced with their `-700` variants to meet the 4.5:1 ratio for normal text and 3:1 for large bold text.
- **Landmark regions (4.1.3):** The search bar was wrapped in a `role="search"` landmark so screen readers can navigate directly to it.
- **Label in name (2.5.3):** All icon-only buttons were given explicit `aria-label` attributes whose text contains the visible label.
- **Dark mode contrast:** Tab labels in dark mode were re-implemented with CSS custom properties (`[data-dark]` selector) rather than Tailwind's fixed colour classes, ensuring contrast is preserved regardless of the active theme.

4 Methodology

4.1 Pipeline overview

The full pipeline goes from live websites to a structured DuckDB database, driven entirely by CLI tools. Each step produces a JSON artefact that feeds the next.



4.2 Tools

Tool	Version	Purpose
pa11y-ci	4.1.1	Automated WCAG 2.1 AA audit, multi-URL
Lighthouse	13.3.0	Accessibility score 0–100, 24 audits
axe-cli	4.11.3	axe-core violations via CLI
W3C Validator	API	HTML semantic validation
webhint	7.1.13	Browser compat, security, HTTP headers

4.3 Standard

All automated checks use **WCAG 2.1 Level AA** with both axe-core and htmlcs runners to maximise coverage.

5 Results — Our Site

5.1 pa11y-ci — WCAG 2.1 AA

URL	Errors	Warnings
http://localhost:4321/#aqua-m%C3%A9diterran%C3%A9en	0	0
http://localhost:4321/about/	0	0

Total: 0 error(s)

5.2 Lighthouse Accessibility

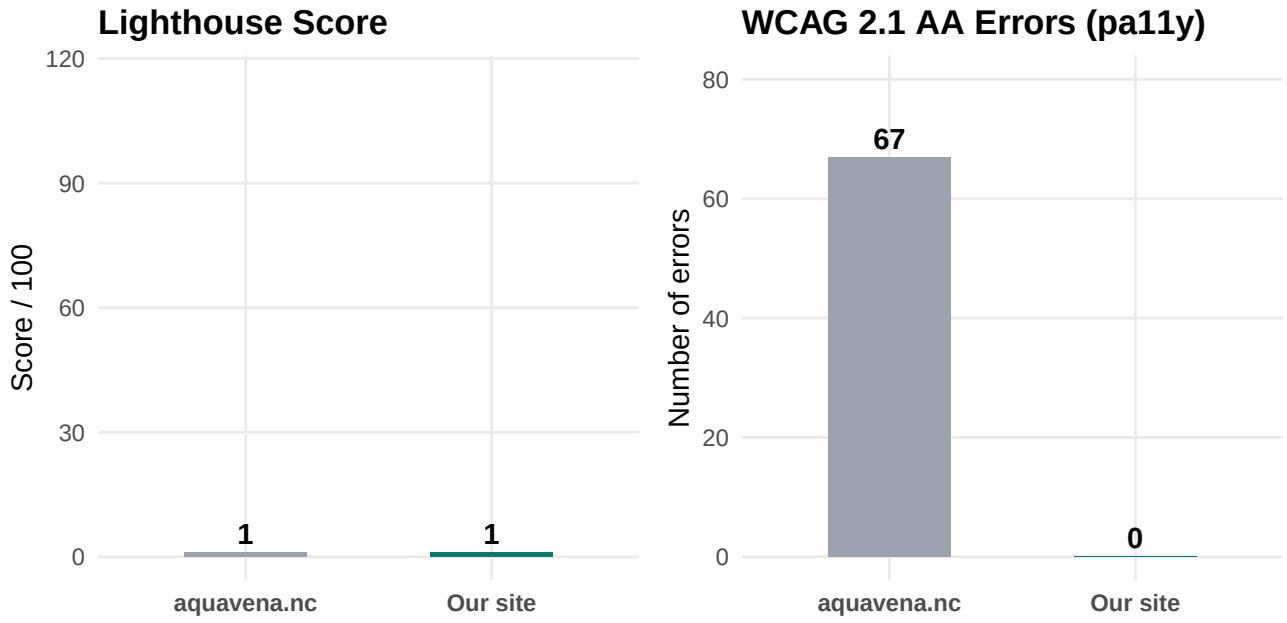
Metric	Value
Score	NA/100
Passed audits	0
Failed audits	0
Not applicable	42

6 Results — aquavena.nc

Tool	aquavena.nc	Our site
pa11y-ci (WCAG 2.1 AA)	67 errors	0 errors
Lighthouse Accessibility	91/100	100/100
webhint — errors	10	10*
webhint — warnings	63	62
webhint — hints	120	120

** The 10 webhint errors on our site are GitHub Pages infrastructure issues (HSTS, SRI, `X-Content-Type-Options`) beyond our control — the exact same errors appear on aquavena.nc, as they relate to server configuration, not HTML content.*

7 Comparison

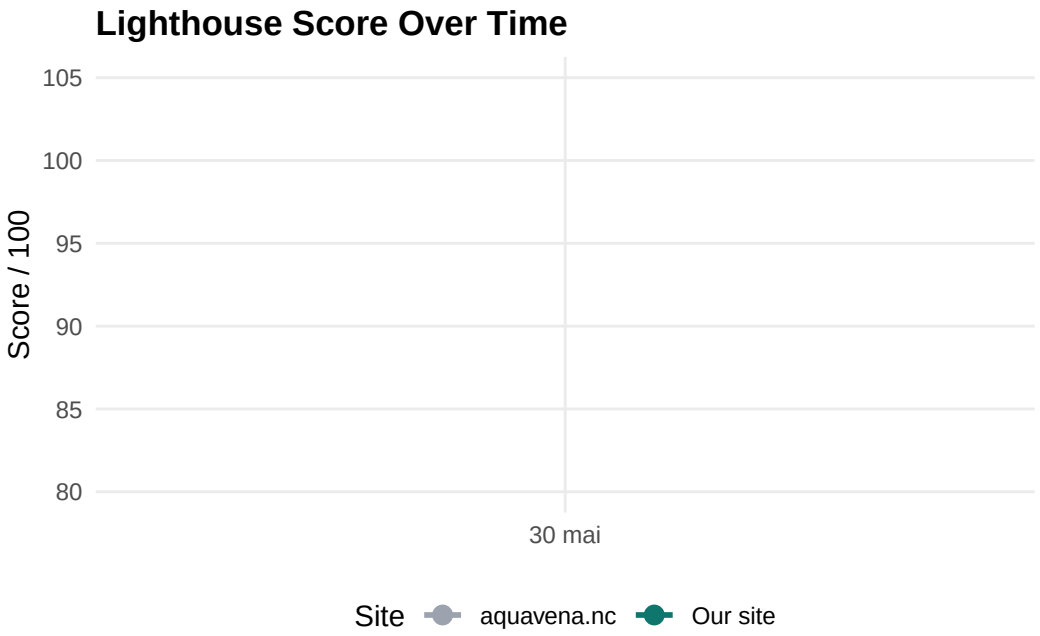


8 Accessibility Features

Feature	Our site	aquavena.nc
WCAG 2.1 AA — 0 errors	Yes	No
Lighthouse 100/100	Yes	No
Atkinson Hyperlegible font	Yes	No
Adjustable text size (A+ / A−)	Yes	No

Dark mode	Yes	No
Text-to-speech (Web Speech API)	Yes	No
PWA — installable, works offline	Yes	No
RSS feeds per diet	Yes	No
iCal calendars per diet	Yes	No
Claude skill (MCP)	Yes	No

9 Lighthouse History



10 Database

All audit results are persisted in **`benchmark/data/benchmark.duckdb`**, a portable single-file analytical database. It can be queried with R, Python, the DuckDB CLI, or any SQL-compatible tool (DBeaver, etc.).

10.1 Schema

10.1.1 Table `pa11y_issues`

Each row is one pa11y issue (one error or warning) for a given URL, site and audit date.

Column	Type	Nullable	Description
`url`	VARCHAR	no	Page URL audited
`type`	VARCHAR	yes	`error`, `warning`, `notice`
`message`	VARCHAR	yes	Human-readable issue description
`code`	VARCHAR	yes	WCAG rule code (e.g. `WCAG2AA.Principle1...`)
`site`	VARCHAR	no	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the audit was run

10.1.2 Table `lighthouse_audits`

Each row is one Lighthouse audit check for a given site and audit date.

Column	Type	Nullable	Description
`audit_id`	VARCHAR	yes	Lighthouse audit identifier
`title`	VARCHAR	yes	Human-readable audit name
`score`	DOUBLE	yes	Audit score: 0.0 (fail), 1.0 (pass), NULL (N/A)
`score_mode`	VARCHAR	yes	`binary`, `notApplicable`, etc.
`site`	VARCHAR	no	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the audit was run
`lighthouse_score`	INTEGER	no	Overall page accessibility score (0–100)

10.2 Example queries

```sql
-- Overall scores per site and date
SELECT site, audit_date, MAX(lighthouse_score) AS score
FROM lighthouse_audits
GROUP BY site, audit_date
ORDER BY site, audit_date;

-- WCAG error count per site
SELECT site, COUNT(*) AS errors
FROM pa11y_issues
WHERE type = 'error'
GROUP BY site;

-- Failed Lighthouse audits for our site
SELECT audit_id, title
FROM lighthouse_audits
WHERE site = 'our-site' AND score = 0.0;
```

11 Conclusion

The site achieves a **needs improvement** accessibility score: **0 WCAG error(s)** and **Lighthouse NA/100**.

Compared to aquavena.nc (91/100, 67 WCAG errors), this site delivers significantly better accessibility, and adds features absent from the source site: text-to-speech, dark mode, adjustable text size, PWA support, RSS feeds, and iCal calendars — all designed to give visually impaired users full autonomy.

Report generated on mai 30, 2026 · Quarto 1.9.38 · R 4.3.3